Digital Wallet & Ledger System

PostgreSQL + Node.js Financial Backend

Bekzhan Kasymbekov

American University of Central Asia

kasymbekov_be@auca.kg

American University of Central Asia

May 2026

Project Objective

Goal: Build a production-style financial backend system using PostgreSQL and Node.js.

The system demonstrates:

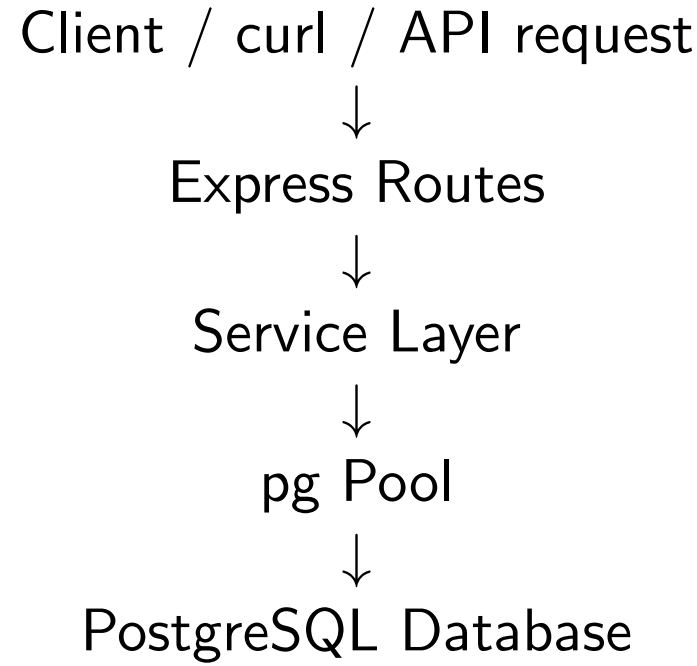
- normalized relational schema design
- ledger-based accounting
- transaction safety
- concurrency control
- indexing and query analysis
- backend integration with Express and PostgreSQL

Technology Stack

- **Database:** PostgreSQL
- **Backend:** Node.js + Express
- **Database Driver:** pg
- **Configuration:** dotenv
- **Environment:** Ubuntu Linux
- **Testing:** psql, curl, jq

Main idea: PostgreSQL is responsible for data integrity, while Node.js handles API logic and transaction flow.

System Architecture



- Routes validate input and return responses.
- Services contain business logic.
- PostgreSQL stores users, accounts, transactions, and ledger entries.

Core Database Entities

- **users**
  - stores user accounts and unique emails
- **accounts**
  - stores wallet accounts owned by users
- **transactions**
  - stores logical financial operations
- **ledger_entries**
  - stores actual money movement

Important: transactions do not store amounts. Money movement is stored in ledger entries.

ER Relationships

users	1 : many	accounts
accounts	1 : many	ledger_entries
transactions	1 : many	ledger_entries

Meaning:

- One user can have many accounts.
- One account can have many ledger entries.
- One transaction can create one or more ledger entries.

For example, a transfer creates two ledger entries.

Ledger-Based Balance Model

The system does **not** store balance directly.

Balance is derived from:

```
SELECT COALESCE(SUM(amount), 0) AS balance
FROM ledger_entries
WHERE account_id = $1;
```

- Positive amount = credit
- Negative amount = debit
- Ledger entries are the source of truth

$$\text{balance} = \sum \text{ledger\_entries.amount}$$

Ledger Rules

Deposit

- creates one transaction
- creates one positive ledger entry

$$+100.00$$

Withdrawal

- creates one transaction
- creates one negative ledger entry
- checks balance before inserting

$$-50.00$$

Transfer

- creates one transaction
- creates two ledger entries

$$25.00 + 25.00 = 0.00$$

Transaction Safety

Each financial operation runs inside a PostgreSQL transaction.

```
BEGIN;

-- lock account
-- check balance
-- insert transaction
-- insert ledger entries
-- update status

COMMIT;
```

If anything fails:

```
ROLLBACK;
```

This prevents partial financial operations from being saved.

Concurrency Control

Withdrawals and transfers use row-level locking:

```
SELECT id
FROM accounts
WHERE id = $1 AND is_active = TRUE
FOR UPDATE;
```

For transfers, both accounts are locked in stable order:

```
SELECT id
FROM accounts
WHERE id IN ($1, $2)
  AND is_active = TRUE
ORDER BY id
FOR UPDATE;
```

This prevents two simultaneous operations from spending the same balance.

System Invariants

The system enforces these rules:

- Every account belongs to a user.
- Every ledger entry belongs to an account.
- Every ledger entry belongs to a transaction.
- Ledger entry amount cannot be zero.
- Withdrawals cannot create negative balances.
- Transfers must conserve money.
- Each transaction has a unique reference ID.

Example:

$$\text{transfer sum} = 0$$

Indexes

Indexes were added for common lookup patterns.

```
CREATE INDEX IF NOT EXISTS idx_ledger_entries_account_id
ON ledger_entries(account_id);

CREATE INDEX IF NOT EXISTS idx_ledger_entries_account_created_at
ON ledger_entries(account_id, created_at DESC);

CREATE INDEX IF NOT EXISTS idx_transactions_type
ON transactions(transaction_type);
```

Used for:

- balance lookup
- account transaction history
- filtering transactions by type

EXPLAIN ANALYZE Results

A modest large seed was created:

- 100 users
- 100 accounts
- 3100 transactions
- 4100 ledger entries

Example result:

- Balance lookup used an index scan.
- Transaction type filtering used an index scan.
- Status filtering used a sequential scan because all rows had status `completed`.

Lesson: indexes are most useful when they filter out many rows.

API Endpoints

Users

- `POST /users`
- `GET /users`

Accounts

- `POST /accounts`
- `GET /accounts/:id/balance`
- `GET /accounts/:id/transactions`

Transactions

- `POST /transactions/deposit`
- `POST /transactions/withdraw`
- `POST /transactions/transfer`

Example API Request

Example transfer request:

```
curl -s -X POST http://localhost:3000/transactions/transfer \
  -H "Content-Type: application/json" \
  -d '{
    "sender_account_id": 1,
    "receiver_account_id": 2,
    "amount": 25,
    "reference_id": "api-transfer-alice-bob-001"
  }' | jq
```

This creates:

- one transaction row
- one negative ledger entry
- one positive ledger entry

Backup and Restore

Backup:

```
pg_dump -h localhost -U postgres -d wallet_system \
  -F c -f backups/wallet_system.backup
```

Restore:

```
createdb -h localhost -U postgres wallet_system_restore

pg_restore -h localhost -U postgres \
  -d wallet_system_restore backups/wallet_system.backup
```

The restored database was verified by checking tables and account balances.

Conclusion

The project implements a functional ledger-based wallet backend.

Implemented features:

- normalized PostgreSQL schema
- ledger-based balance calculation
- deposits, withdrawals, and transfers
- transaction safety and rollback
- concurrency control
- indexing and query analysis
- backup and restore documentation
- Express API integration

Main takeaway: Financial correctness depends on the ledger, transactions, and database constraints.

Thank you!

Questions?